Quantum-Resistant Password-Authenticated Searchable Encryption in Practice

A reproduction of QPASE and a forward-private counter-based search index

Iman Kalyan Chakraborty
Roll No. 2026801003

[Partner name]
Roll No. [Roll number]

RIS Course Project Report

November 9, 2026

Abstract

Searchable encryption lets a user store encrypted files on cloud servers and still search them by keyword. Most schemes need a long secret key on every device the user owns. Password-authenticated searchable encryption (PASE) replaces that key with a password, and QPASE, by Jiang and Wang, rebuilds PASE on lattices so that it resists quantum attacks [1]. In this project we implemented QPASE in Python, following the paper phase by phase: the distributed key generation, the lattice threshold oblivious pseudorandom function with its robust extractor, Register, Login, Outsource, Retrieve and the server key update. Where the paper is incomplete or inconsistent, we record the choice we made. Our implementation passes 184 unit tests, and on one machine it recovers the same user key in 10/10 logins and returns exactly the matching documents for a real keyword query. Building it uncovered two gaps. Run in the ring $\mathbb{Z}_q[X]/(X^{595}+1)$, the paper's 128-bit parameter set lets one server test password guesses offline, and the linear commitments of the key update reveal every update share. We then propose a counter-based search index that keeps the search key on the user's device. Servers answer a search with dictionary lookups instead of a scan, and a stored search request cannot find files uploaded after it (forward privacy). At the slowest server it searched 109 to 152 times faster than the paper's scheme for 10^2 to 10^5 stored files.

To do (Person B). Revise this abstract once Sections 6 to 10 are written, so it matches what they claim.

Keywords: searchable encryption, password authentication, lattice cryptography, oblivious pseudorandom function, forward privacy, post-quantum security.

1 Introduction

1.1 Background

Users keep more and more of their data with cloud providers, and much of it is sensitive: medical records, invoices, travel bookings. A user who encrypts such files before upload protects them from the provider, but loses the ability to search them on the server. Downloading and decrypting everything for each search costs bandwidth and time. Searchable symmetric encryption (SSE) closes this gap. The user attaches an encrypted index to each file, and the server can find the files that match a keyword without learning the keyword or the contents [2, 3].

SSE needs a long, uniformly random key, and that key has to be present on every device the user searches from. People cannot remember such keys, so in practice the key is stored and copied between devices, and each copy is one more place it can leak. A password is what people actually remember. A line of work on password-protected secret sharing lets a user recover a strong key from a password with the help of several servers, none of which learns the password or the key [4, 5, 6, 7, 8]. Chen et al. combined this idea with searchable encryption in password-authenticated searchable encryption (PASE) [9, 10]. These

schemes rest on the discrete-logarithm problem, which Shor's algorithm solves on a quantum computer [11]. Lattice-based schemes avoid that attack [12, 13, 14]. QPASE is the first PASE scheme built on lattices [1]: the user remembers only an identity and a password, a threshold of t out of N servers turns the password into a user key K_u, and every other key is derived from K_u.

1.2 Problem statement

The QPASE paper specifies the scheme and reports measurements of a C++ implementation, but it does not include that implementation. A reader who wants to check the scheme's claims, or to build on it, has to implement it first. That raised three questions for this project:

1. Can QPASE be built from the paper alone, and where does the paper leave a choice open?
2. What does the scheme cost in time and traffic when it is built that way, and does the paper's design hold up when each step runs?
3. The paper's search sends a permanent per-keyword key to every server, and each server tests that key against every file it stores. Can search keep the key on the user's device and touch only the matching files?

1.3 Contributions

- **A working QPASE.** We implemented every phase of the paper in Python: the distributed key generation, the lattice threshold oblivious pseudorandom function (TOPRF) with its robust extractor, Register, Login, Outsource, Retrieve and the server key update (Sections 3 and 4). Eight notebooks show each phase with the data that crosses the wire, and 184 unit tests check the behaviour.
- **A record of every gap in the paper.** Where the paper is incomplete or internally inconsistent, we state what it says, what we chose and why (Section 4.3).
- **Measured costs on one machine.** We time every building block and every phase at the user and at a server, explain each phase time by its count of lattice PRF evaluations, measure exact message sizes, and compare both with the paper's own figures (Section 5).
- **Two security findings.** Run in the ring $\mathbb{Z}_q[X]/(X^{595}+1)$, the paper's 128-bit parameter set lets a single server test password guesses offline. The commitments in the server key update reveal every update share to an eavesdropper (Section 5.6).
- **A counter-based search index.** We propose and evaluate an index that keeps the search key on the user's device, answers a search with dictionary lookups, and hides later uploads from earlier searches (Sections 6 to 8).

1.4 Organisation

Section 2 reviews related work and the building blocks. Section 3 describes QPASE as the paper specifies it, and Section 4 describes our implementation and the choices it required. Section 5 evaluates the implementation and reports the two findings. Section 6 proposes the counter-based index, Section 7 analyses its security, and Section 8 measures it. Section 9 compares the schemes, and Section 10 concludes. Section A explains how to reproduce every number in this report.

2 Literature Review and Preliminaries

2.1 Related work

Searchable encryption. Song, Wagner and Perrig gave the first practical scheme for searching encrypted data [2]. Curtmola et al. defined the security of searchable symmetric encryption and built schemes with an encrypted index [3]. Later work added conjunctive and other richer queries [15]. All of these assume that the user holds a long secret key.

Keys from passwords. Password-protected secret sharing (PPSS) lets a user recover a secret from a password and a threshold of servers [4]. Jarecki et al. made it round-optimal and then built it from a threshold oblivious pseudorandom function (OPRF) [5, 6, 7]. DPaSE derives many keys from one password [8]. Password-authenticated keyword search and PASE apply the same idea to searchable encryption [10, 9]. All of these rely on groups where the discrete logarithm is hard, which a quantum computer breaks [11].

Quantum resistance. Roy et al. built PPSS from lattices [13]. Jiang et al. built a lattice threshold single-sign-on scheme with updatable server keys [16] and a password-protected outsourcing scheme, QPause [14]. QPASE adds keyword search, which gives the first password-authenticated searchable encryption scheme that resists quantum attacks [1].

2.2 Notation

Table 1 lists the notation used in the rest of the report. It follows the QPASE paper.

Table 1: Notation.

Symbol	Meaning
$R_q = \mathbb{Z}_q[X]/(X^n+1)$	polynomial ring, modulus q, degree n
$\ell = \lceil \log_2 q \rceil$	width of the gadget vector
χ_σ	discrete Gaussian distribution with standard deviation σ
$a, a_0, a_1 \in R_q^{1\times\ell}$	public vectors from Setup
$G = (1,2,\ldots,2^{\ell-1})$, G^{-1}	gadget vector and binary decomposition
N, t	number of servers and threshold
S_i, k_i, pk_i	server i, its share of the master key K, its public key
U, ID_u, psw_u	user, identity and password
$x \in \{0,1\}^L$	binary encoding of the password
K_u	user key output by the TOPRF
(pk_u, sk_u)	user's signature key pair, derived from K_u
w, $dsk = \mathrm{HKDF}(K_u, w)$	keyword and its search key
C, Ct, i	encrypted document, its index ciphertext, its item id
μ, μ_u	limit on login attempts per epoch, and the user's current count
λ_i	Lagrange coefficient of server i at 0

2.3 Ring learning with errors

All lattice operations in QPASE take place in R_q. An element is a polynomial of degree below n with coefficients in $\mathbb{Z}_q$, and a product wraps around with $X^n = -1$. Security rests on the decision learning-with-errors problem [12] in its ring form [17].

Definition 1 (Decision ring-LWE)**.** *For a secret $s \leftarrow \chi_\sigma$, samples $(a, as+e)$ with a uniform in R_q and $e \leftarrow \chi_\sigma$ are computationally indistinguishable from pairs (a,b) with b uniform in R_q.*

2.4 Lattice pseudorandom function

QPASE uses the Banerjee–Peikert pseudorandom function (PRF) [18]. The gadget vector $G = (1, 2, \ldots, 2^{\ell-1})$ and its inverse G^{-1}, the bit decomposition of each coefficient, turn a vector in $R_q^{1\times\ell}$ into an $\ell \times \ell$ matrix of binary polynomials with $G \cdot G^{-1}(v) = v$. For an input $x = (x_1, \ldots, x_L) \in \{0,1\}^L$ and the public vectors a_0, a_1,

$$a_x = a_{x_1}\, G^{-1}\big(a_{x_2}\, G^{-1}(\cdots G^{-1}(a_{x_L}))\big), \qquad F_k(x) = \big\lfloor (p/q) \cdot a_x \cdot k \big\rceil. \tag{1}$$

Computing a_x takes $L-1$ steps, each a product of a $1 \times \ell$ vector with an $\ell \times \ell$ matrix of ring elements. The rounding to $\mathbb{Z}_p$ is the learning-with-rounding idea [19]. We call one evaluation of a_x a *PRF chain*. Section 5 shows that this chain dominates the cost of the whole scheme.

2.5 Threshold OPRF with a robust extractor

The TOPRF lets the user evaluate $F_K(x)$ on its password encoding x under the master key K, which no single party holds [20, 16]. Server S_i holds a share k_i of K and publishes $pk_i = ak_i + 2e_i$. The protocol has three moves.

1. **Blind.** The user samples $r, e \leftarrow \chi_\sigma$ and sends $b = ar + a_x + 2e$ to at least t servers. The term $ar + 2e$ hides a_x, and therefore the password.
2. **Evaluate.** Each contacted server returns $b_{k_i} = bk_i + 2e'_i$.
3. **Combine.** With the Lagrange coefficients λ_i of the contacted servers, the user computes

$$F = \sum_i \lambda_i b_{k_i} - r \sum_i \lambda_i pk_i = a_x K + 2e''. \tag{2}$$

The noise $2e''$ is even and small, but it changes from one login to the next, so rounding F alone would give a slightly different key each time. QPASE removes it with the robust extractor of Ding et al. [21]. For a coefficient c in its centred representation, two signal functions mark whether c lies near the boundary between the two halves of $\mathbb{Z}_q$:

$$\sigma_0(c) = \begin{cases} 0 & c \in [-\lfloor q/4 \rfloor, \lfloor q/4 \rfloor] \\ 1 & \text{otherwise} \end{cases}, \qquad \sigma_1(c) = \begin{cases} 0 & c \in [-\lfloor q/4 \rfloor + 1, \lfloor q/4 \rfloor + 1] \\ 1 & \text{otherwise.} \end{cases}$$

The hint algorithm $S(y)$ outputs $\sigma_b(y)$ for a random bit b, and the extractor is $E(x, \sigma) = \big((x + \sigma \cdot \frac{q-1}{2}) \bmod q\big) \bmod 2$. If $x - y$ is even and small, then $E(x, S(y)) = E(y, S(y))$. The user key is $K_u = E(F, \sigma)$, one bit per coefficient.

2.6 Distributed key generation

The servers create the master key K without a trusted dealer [22]. Each server S_i deals a Shamir sharing of a random contribution s_i to all servers, and server S_j sums the shares it receives to get k_j. The master key $K = \sum_i s_i$ is never assembled anywhere, but any t servers can use it through their shares, and fewer than t learn nothing about it.

2.7 Symmetric and signature primitives

HKDF and PRF. QPASE derives a search key per keyword with an extract-then-expand key derivation function, $dsk = \mathrm{HKDF}(K_u, w)$ [23], and tags each document with $v = \mathrm{PRF}(dsk, \rho)$ for a random ρ.

Signatures. Every message after Login is signed with CRYSTALS-Dilithium [24], standardised by NIST as ML-DSA [25]. The user's key pair is derived from K_u, so it is never stored.

Symmetric encryption. Documents and index entries are encrypted with AES-256. Grover's algorithm halves its effective key length [26], which still leaves 128 bits of security. We use the authenticated Galois/Counter Mode [27].

3 The QPASE Scheme

This section restates QPASE as Sections III and IV of the paper specify it [1]. Where our implementation departs from the printed text, Section 4.3 says so.

3.1 System and threat model

QPASE has one kind of user and a set of N cloud servers $S = \{S_1, \ldots, S_N\}$.

- **User U.** The user remembers only ID_u and psw_u, and stores no key on any device. At every Login it rebuilds K_u with t servers and derives everything else from it.
- **Servers.** Each server holds a share k_i of the master key K, a record (ID_u, pk_u, μ_u) per user, and the user's encrypted files. The servers are semi-honest and talk to each other over secure channels.
- **Adversary.** A quantum adversary controls the network and may corrupt up to $t' < t$ servers. It tries three attacks: offline password guessing, online password guessing, and chosen-keyword attacks against the search index.

The paper proves authentication security in the Bellare–Pointcheval–Rogaway model [28], with passwords following Zipf's law [29], and keyword privacy as indistinguishability under chosen-keyword attacks (IND-CKA). The servers count login attempts per user and refuse after μ in one epoch, which bounds online guessing.

3.2 Setup

Setup publishes the parameters pp: the ring R_q, the public vectors $a, a_0, a_1 \in R_q^{1\times\ell}$, the Gaussian width σ, the attempt limit μ, the numbers N and t, and the hash, HKDF, PRF, encryption and signature algorithms. The servers then run the distributed key generation, so that S_i holds k_i and publishes pk_i. Table II of the paper fixes $q = 2^{28} - 57$ and two ring degrees: PARAM I with $n = 512$, rated at 92-bit quantum security, and PARAM II with $n = 595$, rated at 128-bit quantum security. The paper's experiments use PARAM II.

3.3 Register

Register (paper Section IV-B, Fig. 5)

1. U encodes psw_u as $x \in \{0,1\}^L$ and runs the TOPRF with every server S_i to obtain $K_u = F_K(x)$. Each server rejects an ID_u it already holds.
2. U derives its signature key pair $(pk_u, sk_u) \leftarrow \mathrm{Gen}(1^\kappa, K_u)$ and sends pk_u to every S_i.
3. Each S_i stores (ID_u, pk_u) next to its share k_i and sets the attempt counter $\mu_u = 0$.
4. U keeps only ID_u and psw_u.

3.4 Login

Login (paper Section IV-C, Fig. 5)

L1. U runs the TOPRF on x with t servers. Each contacted S_i first checks $\mu_u < \mu$ and aborts otherwise, then sets $\mu_u \leftarrow \mu_u + 1$, answers, and returns pk_u.

L2. U recomputes (pk'_u, sk'_u) from K_u. If $pk'_u \neq pk_u$ the password was wrong, and U asks for it again. Otherwise U signs $\sigma_u \leftarrow \mathrm{Sign}(sk'_u, (ID_u, sid))$ for the session identifier sid.

L3. S_i verifies σ_u under pk_u. On success it lets U upload and search. On failure it aborts and sets $\mu_u \leftarrow \mu_u + 1$.

A wrong password is caught at step L2 on the user's side, without any extra round. The server never sees the password, the user key or the signing key.

3.5 Outsource

Outsource (paper Section IV-D, Fig. 6)

O1. For a document d with keyword w and item id i, U samples $\rho \leftarrow \mathbb{Z}_q$ and computes

$$dsk = \mathrm{HKDF}(K_u, w), \qquad v = \mathrm{PRF}(dsk, \rho), \qquad C = \mathrm{Enc}(K_u, d),$$
$$\sigma_C = \mathrm{Sign}(sk_u, (\rho, v, C)), \qquad Ct = \mathrm{Enc}(dsk, (\rho, v, \sigma_C)), \qquad \sigma_{Ct} = \mathrm{Sign}(sk_u, (Ct, C, i)).$$

U sends (Ct, C, i) and σ_{Ct} to a server of its choice.

O2. The server verifies σ_{Ct} under pk_u and, if it holds, stores (Ct, C, i). Each server offers the user M storage slots.

3.6 Retrieve

Retrieve (paper Section IV-E, Fig. 6)

R1. U computes $dsk = \mathrm{HKDF}(K_u, w)$ and $\sigma_{dsk} = \mathrm{Sign}(sk_u, dsk)$ and sends both to every server.

R2. Each S_i verifies σ_{dsk}. For every stored (Ct, C, i) it decrypts $(\rho, v, \sigma_C) = \mathrm{Dec}(dsk, Ct)$ and adds the item to its list L_i if $\mathrm{Ver}(pk_u, (\rho, v, C), \sigma_C)$ holds and $v = \mathrm{PRF}(dsk, \rho)$. It returns L_i.

R3. U repeats the same check on each returned item and decrypts $d = \mathrm{Dec}(K_u, C)$.

The server finds a match by trying dsk on every item it stores. Items under other keywords fail at the decryption. The user sends the same dsk every time it searches for w.

3.7 Server key update

Server key update (paper Section IV-F)

1. Each S_j chooses a random polynomial $[F]_j(X) = \sum_{k=1}^{t-1} \alpha_k X^k$ with constant term 0.
2. S_j computes commitments $h = \{H(\alpha_k)\}$ and the shares $F_{ji} = [F]_j(i)$.
3. S_j sends each S_i the share F_{ji}, encrypted to S_i, with h, its epoch ω and a signature.
4. S_i decrypts each share and checks the signature and $H(F_{ji}) = \sum_k i^k H(\alpha_k)$. It then updates $k'_i = k_i + \sum_j \lambda_{i,j} [F]_j(i)$.

5. S_i recomputes pk_i', resets every μ_u, and starts epoch $\omega + 1$.

Because every update polynomial has constant term 0, the master key K does not change (the paper's Lemma 3). Users therefore keep their K_u and their stored files, and need not take part in the update.

3.8 Multiple keywords

Section IV-G of the paper sketches an extension to several keywords per document, with conjunctive, disjunctive and subset queries. It lies outside the paper's main protocol, its proofs and its experiments, and we left it out of this project.

4 Implementation

4.1 Scope and structure

We implemented QPASE in Python 3.14.9 with three libraries: numpy for the ring arithmetic, `cryptography` for AES-256-GCM, and `dilithium-py` for ML-DSA-44. The implementation covers Setup, the distributed key generation, Register, Login, Outsource, single-keyword Retrieve and the server key update. The notebooks call the library and add no protocol code of their own. Table 2 lists the modules.

Table 2: Modules of the implementation, all under `src/ris_sem_project/`.

Module	Responsibility
`params.py`	the Table II parameter sets and Setup; parses pp once
`primitives/ring.py`	exact arithmetic in R_q
`primitives/sampling.py`	uniform sampling and the discrete Gaussian χ_σ
`primitives/gadget.py`, `bp_prf.py`	G^{-1} and the Banerjee–Peikert PRF of Equation (1)
`primitives/extractor.py`	Ding's hint S and extractor E
`primitives/secret_sharing.py`	Shamir sharing over R_q, Lagrange coefficients, the key generation
`primitives/toprf.py`	blinding, evaluation and the combination of Equation (2)
`primitives/hkdf.py`, `prf.py`	HKDF and PRF built on the lattice PRF
`primitives/signature.py`, `symmetric.py`	ML-DSA-44 and AES-256-GCM wrappers
`primitives/linear_hash.py`	the linear hash H of the key-update commitments
`index.py`	builds the index ciphertext Ct and runs the check of R2 and R3
`parties/`	the user and server classes and the wire messages
`protocol.py`	runs each phase and records every message it moves
`root_attack.py`	the PARAM II offline password test of Section 5.6.1
`bench.py`, `figures.py`	measurement code and the figures; no protocol logic

Every value that crosses between parties is an immutable record whose cryptographic fields are bytes. The user and server classes check those bytes when they arrive and trust their own internal types afterwards. The same function checks an index entry at the server in step R2 and at the user in step R3, so the two sides cannot drift apart. The user object holds one session state at a time (a TOPRF in flight, a Register waiting to send its key, or a logged-in session), which makes a call in the wrong order an error rather than a silent mistake.

4.2 Exact ring arithmetic

The modulus $q = 2^{28} - 57$ is not suitable for the number-theoretic transform, since $q - 1 = 2 \cdot 134217699$ has no large power of two as a factor. We multiply in R_q with a twisted complex FFT instead. Coefficient i of each operand is multiplied by ψ^i with $\psi = e^{i\pi/n}$, which turns the wrap-around $X^n = -1$ into an ordinary

cyclic convolution. To keep the floating-point result exact, each operand is split into two 14-bit limbs. Every limb product then has integer coefficients below $n \cdot 2^{29}$, well inside the 53-bit precision of a double, so rounding recovers each coefficient exactly. This works for any n, including the odd $n = 595$ of PARAM II. The tests compare the product with a schoolbook implementation, and notebook 01 finds 3/3 random products equal to an exact integer reference.

4.3 Where the paper is incomplete

The paper leaves several values open and contradicts itself in a few places. Table 3 records each gap and the choice we made. The implementation plan in `docs/implementation-plan.md` gives the full reasoning.

Table 3: Gaps in the paper and our choices.

Topic	What the paper says	Our choice
Ring or matrices	Fig. 1 uses LWE matrices; Section II-B, Section IV-A and Fig. 2 use a vector in $R_q^{1\times\ell}$.	The ring $R_q = \mathbb{Z}_q[X]/(X^n+1)$. The matrix version of the PRF would need a 14336×14336 matrix product per step at $n = 512$.
TOPRF combination	The proof of Theorem 1 prints $+r$ on one page and $-r$ on another.	The sign the algebra gives, $-r$ in Equation (2).
Lagrange coefficients	Not discussed. The noise e'' is small only if every λ_i is an integer.	Only server subsets whose coefficients are integers. The subset $\{1,\ldots,t\}$ always qualifies; $\{1,3\}$ with $t=2$ does not and is refused.
Key sizes	The key generation leaves the distribution of the contributions open.	Contributions, r and e drawn from χ_σ, so K stays small as the TOPRF requires.
Extractor hint	Fig. 2 computes the hint from vk_i, a value unrelated to F.	The user computes $\sigma = S(F)$ at Register; every server stores it and returns it at Login. Two logins then differ by an even, small amount, and K_u repeats.
Sharing degree	Definition 11 uses degree t, so $t+1$ servers reconstruct, but Login contacts t.	Degree $t-1$, so any t servers suffice.
Key-update check	$H(F_{ji}) = \sum_k i^k H(\alpha_k)$ needs a linear H, and the update weights the shares by $\lambda_{i,j}$.	$H(x) = A_H x \bmod q$ with A_H expanded from SHAKE256, and $k_i' = k_i + \sum_j F_{ji}$, which keeps K unchanged.
Secure channels	Assumed between servers.	AES-256-GCM under a key per server pair, and an ML-DSA-44 key per server for the share signatures.
Open values	p, σ, L, the encryption mode and the Dilithium level are not fixed.	$p=2$, $\sigma=3.2$, $L=128$ (the first bits of SHA3-256 of the password), AES-256-GCM and ML-DSA-44.
Meaning of σ	The text calls it the standard deviation; the printed density makes it a width.	The standard deviation, as the text says.
HKDF and PRF	HKDF "introduces the lattice PRF into Krawczyk's framework".	Extract and expand both evaluate the lattice PRF. The Dilithium seed and the AES key also come from HKDF of K_u.
Authorisation	L3 "allows" the user to upload and search.	Outsource and Retrieve are authorised by σ_{Ct} and σ_{dsk}. Retrieve reaches all N servers but Login only t, so an L3 gate would lock the other servers out.
Attempt counter	L1 increments μ_u on every attempt; only the key update resets it.	As printed. A user therefore gets μ logins per epoch in total, not μ failed ones.
Multiple keywords	Section IV-G sketches it outside the main protocol.	Left out; queries take one keyword.

The sampler for χ_σ and the extractor are not constant-time. That is acceptable for a study of the scheme, but a deployment would need constant-time versions.

4.4 How we checked the implementation

We checked the implementation at three levels.

- **Unit tests.** 184 tests check behaviour that a caller can observe: exact ring products against a schoolbook reference, K_u identical across logins and server subsets, wrong passwords and tampered messages rejected, documents recovered byte for byte, and the master key unchanged by a key update.
- **Notebooks.** Eight notebooks run each phase with real data and print every message. Each ends with assertion cells, so a notebook that finishes without an error means the property it demonstrates held.
- **One run, one machine.** Every number in Sections 5 and 8 comes from one run of all eight notebooks on the machine of Table 4. A script checks each number that the repository's README quotes against the notebook cell that printed it, and the numbers in this report are generated from the same cells (Section A).

5 Evaluation of the Reproduced Scheme

5.1 Setup

All measurements ran on the machine in Table 4, and Table 5 gives what each notebook cost to run on it. The protocol notebooks use PARAM I with $N = 3$ servers and threshold $t = 2$. Section 5.6.1 explains why we do not use PARAM II. The cost notebook pins its process to one performance core and runs each measured item once as a warm-up and then seven times (31 or 101 times for the cheapest building blocks). We report the median and the range. Every timed run also checks its own result, and all 270 checked runs gave the right answer (0 failures). Timings belong to this machine. Message sizes, counts and pass or fail results are the same on any machine.

Table 4: Test machine.

Item	Value
Processor	13th Gen Intel(R) Core(TM) i5-13500, 20 logical CPUs
Memory	15.3 GiB
Operating system	Arch Linux, kernel Linux 7.2.11-arch1-1 (x86_64)
Python	3.14.9
Packages	numpy 2.5.3, cryptography 50.0.2, dilithium-py 1.4.0, ipykernel 7.4.0, nbclient 0.11.0, matplotlib 3.11.2

Table 5: Cost of running each notebook on the test machine. Wall time and peak memory are those of the notebook's own kernel, printed by its last cell.

Notebook	Wall time	Peak memory
01 preliminaries	9 s	117 MiB
02 register and login	35 s	118 MiB
03 outsource and retrieve	18 s	114 MiB
04 server key update	21 s	121 MiB
05 costs	141 s	148 MiB
06 PARAM II attack	43 s	135 MiB
07 counter index	22 s	117 MiB
08 search scaling	828 s	574 MiB

5.2 Functional results

Each item below is printed by a notebook and checked by its assertion cells.

- **The TOPRF noise stays small.** In one run the largest coefficient of the noise $2e''$ was 5904, and two TOPRF outputs for the same password differed by at most 9194. Both are tiny next to the extractor's bound $q/4 = 67108849$, so the extracted key cannot change.
- **The same password gives the same key.** 10/10 repeated logins produced a byte-identical K_u and pk_u. Logging in through the server subsets $\{1,2\}$ and $\{2,3\}$ gave the same key as Register.
- **Wrong passwords and blocked subsets fail.** A wrong password is caught at step L2, and each contacted server counts the attempt. Once a server's counter reaches $\mu = 14$ it refuses L1 even for the right password. The subset $\{1,3\}$, whose Lagrange coefficients are not integers, is refused before any server is contacted.
- **Search returns exactly the matching documents.** Notebook 03 stores seven real documents on three servers. A search for `medical` returns items [1, 3, 6], byte-equal to the originals. A keyword with no documents returns nothing. A server that flips a bit of an index ciphertext, or swaps a stored document, cannot get the altered item past the checks of R2 and R3.
- **The key update leaves users unaffected.** 9 of 9 update shares pass the checks, every k_i and pk_i changes, and the master key stays the same. After two updates (epochs [2, 2, 2]) the user logs in with the same K_u and retrieves documents stored before the first update. Replayed and forged shares are rejected, and the receiving server's state does not change.

5.3 Computation cost

Figure 1 shows the time of each phase at the user and at one server, and Tables 6 and 7 give the numbers. The user's phases take between 994 ms and 1.90 s. The server's phases take a few milliseconds, except Retrieve and the key update.

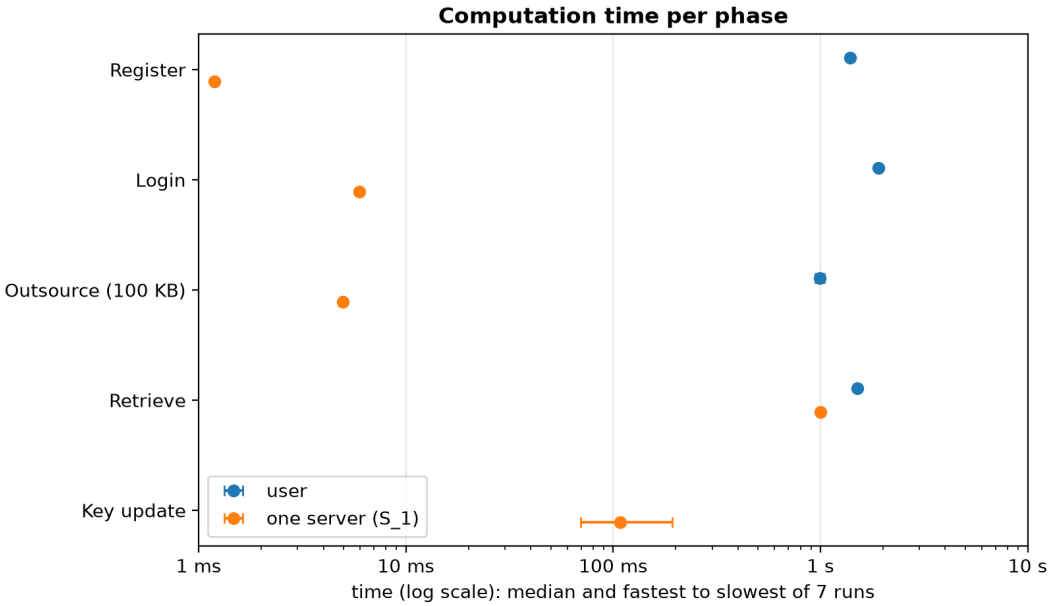


Figure 1: Computation time per phase at the user and at one server, on a log scale: the median of 7 runs and the fastest to the slowest run. Drawn by `notebooks/05_costs.ipynb`, cell 13.

One number explains most of these times. A PRF chain, one evaluation of a_x, takes 492 ms. Every HKDF and PRF call in our implementation is a chain, and so is the password encoding in the TOPRF. The

Table 6: Building blocks. a_x is one PRF chain, Equation (1). Source: `notebooks/05_costs.ipynb`, cell 13.

building block	median	range (min to max)	runs
a_x	492 ms	490 ms to 496 ms	7
Ring.mul (ell, n) x (n,)	0.67 ms	0.66 ms to 0.71 ms	31
ML-DSA-44 keygen	3.94 ms	3.85 ms to 4.01 ms	101
ML-DSA-44 sign	19.8 ms	10.1 ms to 85.6 ms	101
ML-DSA-44 verify	4.79 ms	4.71 ms to 5.37 ms	101
AES-256-GCM encrypt 100 KB	0.021 ms	0.021 ms to 0.023 ms	31
AES-256-GCM decrypt 100 KB	0.021 ms	0.020 ms to 0.022 ms	31
linear hash H	0.20 ms	0.20 ms to 0.21 ms	31
TOPRF blind (U)	1.15 ms	1.14 ms to 1.17 ms	7
TOPRF evaluate (S_i)	1.09 ms	1.09 ms to 1.10 ms	7
TOPRF combine (U)	1.04 ms	1.04 ms to 1.06 ms	7

Table 7: Each phase per party. "PRF chains" counts the lattice PRF evaluations in the phase. "Share in PRF chains" divides the chain count times the a_x time measured right after each run by that run's phase time, so it can read slightly above 100% when a phase is all PRF work. The limiter is the term of the cost model that takes the largest share. Source: `notebooks/05_costs.ipynb`, cell 13.

phase	party	median	range (min to max)	PRF chains	share in PRF chains	limiter (share of time)
Register	user	1.39 s	1.39 s to 1.40 s	3	102%	PRF chains a_x (102%)
Register	one server (S_1)	1.19 ms	1.18 ms to 1.20 ms	0	0%	TOPRF evaluate (92%)
Login	user	1.90 s	1.88 s to 1.91 s	4	97%	PRF chains a_x (97%)
Login	one server (S_1)	5.94 ms	5.87 ms to 5.97 ms	0	0%	ML-DSA-44 verify (81%)
Outsource (100 KB)	user	994 ms	943 ms to 1.05 s	2	93%	PRF chains a_x (93%)
Outsource (100 KB)	one server (S_1)	4.95 ms	4.91 ms to 4.97 ms	0	0%	ML-DSA-44 verify (97%)
Retrieve	user	1.50 s	1.49 s to 1.52 s	3	92%	PRF chains a_x (92%)
Retrieve	one server (S_1)	1000 ms	998 ms to 1.00 s	2	92%	PRF chains a_x (92%)
Key update	one server (S_1)	108 ms	69.6 ms to 193 ms	0	0%	ML-DSA-44 sign (68%)
Key update	all 3 servers	326 ms	271 ms to 433 ms	0	0%	ML-DSA-44 sign (67%)

user's Register runs three chains, Login four, Outsource two, and Retrieve three, and those chains account for over 90% of each phase (Table 7). A server runs no chain except in Retrieve, where it runs one for each item that matches the query. That is why a server's Retrieve takes 1000 ms with two matching items, while its Login takes 5.94 ms, mostly one ML-DSA-44 verification at 4.79 ms. The key update is dominated by ML-DSA-44 signing (19.8 ms median). Encrypting a 100 KB document with AES-256-GCM takes only 0.021 ms.

5.4 Comparison with the paper's reported costs

Table 8 sets our medians next to the paper's Table IV. The paper measured a C++ implementation with NTL on an AMD Ryzen 7 5800H, at PARAM II with $N = t = 2$ [1]. We measured Python at PARAM I with $N = 3$ and $t = 2$. Our user-side Register and Login take more than a thousand times as long as the paper reports, and Outsource and Retrieve about fifty times as long.

Table 8: Time per phase: the paper's Table IV against our medians from Table 7.

	User		Server	
Phase	**Paper**	**Ours**	**Paper**	**Ours**
Register	0.99 ms	1.39 s	0.05 ms	1.19 ms
Login (recover K_u)	0.64 ms	1.90 s	0.05 ms	5.94 ms
Outsource (100 KB)	16.05 ms	994 ms	0.13 ms	4.95 ms
Retrieve	30.95 ms	1.50 s	15.69 ms	1000 ms

Part of the gap is the language: our ring products run through numpy from Python. Most of it is the PRF.

The paper's Table III lists one PRF evaluation at 0.554 ms and one HKDF evaluation at 0.02 ms; our chain takes 492 ms. A full evaluation of Equation (1) needs $(L-1)\,\ell^2$ ring products, which is 99,568 at $L = 128$ and $\ell = 28$, and our profile attributes most of the chain's time to those products. We infer that the paper's PRF and HKDF timings measure a smaller computation, for example a hash-based HKDF, but the paper does not say how it instantiated either function. This matters for the conclusions: in our implementation the lattice PRF, not the TOPRF or the signatures, sets the cost of every user-side phase.

5.5 Communication cost

Figure 2 and Table 9 give the exact bytes each phase sends. The lattice vectors dominate. Each of b, b_{k_i} and pk_i is 57344 B, so Register sends three of them per server and Login three per contacted server. Outsource is dominated by the 100 KB document itself, and Retrieve by the two matching documents that come back.

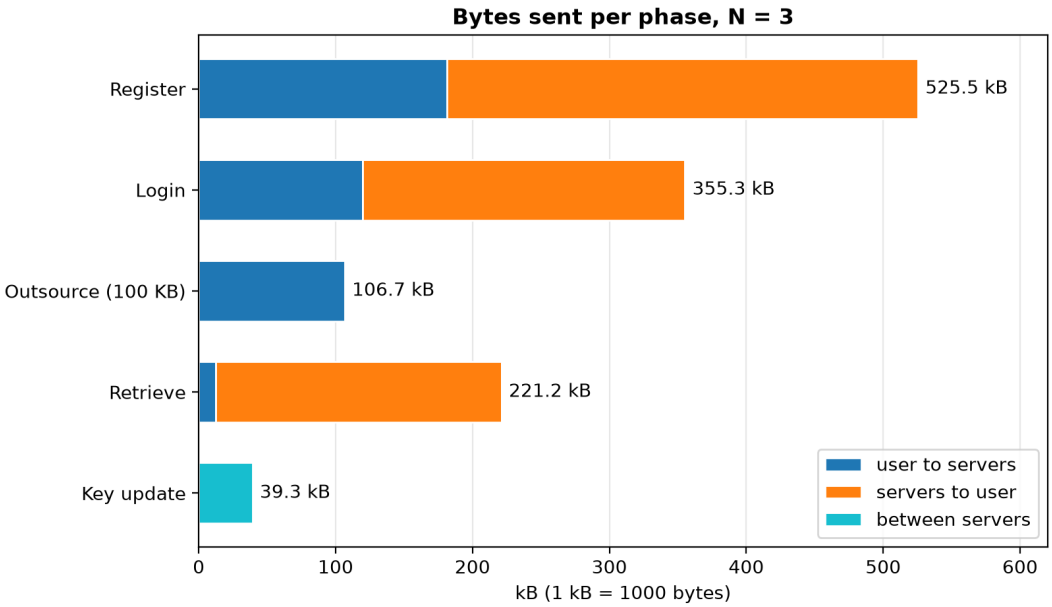


Figure 2: Bytes each phase sends, split by direction, with $N = 3$ servers. Drawn by `notebooks/05_costs.ipynb`, cell 13.

Table 9: Bytes per phase and direction, $N = 3$, $t = 2$, one 100 KB document per Outsource and two matching documents per Retrieve. Source: `notebooks/05_costs.ipynb`, cell 13.

phase	user to servers	servers to user	between servers	total
Register	181398 B	344064 B	0 B	525462 B
Login	119612 B	235648 B	0 B	355260 B
Outsource (100 KB)	106709 B	0 B	0 B	106709 B
Retrieve	12645 B	208568 B	0 B	221213 B
Key update	0 B	0 B	39264 B	39264 B

The paper's Table IV reports about 4,404,019 bits for Register and 2,097,152 bits for recovering K_u, roughly 550 kB and 262 kB, with $N = t = 2$. Our Register (525462 B, $N = 3$) and Login (355260 B, $t = 2$) are of the same order. For Outsource and Retrieve the paper reports about 111,332 and 119,050 bits. 111,332 bits is 13.9 kB, less than the 100 KB file being outsourced, so we read these two figures as bytes; the paper does not say. In bytes they agree with our Outsource of 106709 B.

5.6 Security findings

5.6.1 PARAM II in the ring $\mathbb{Z}_q[X]/(X^{595}+1)$ allows offline password guessing

Table II gives PARAM II the degree $n = 595$. We run it in the ring of the scheme text, $R_q = \mathbb{Z}_q[X]/(X^n+1)$ (Section 4.3). Because 595 is odd, $(-1)^{595}+1 = 0$, so $X = -1$ is a root of $X^{595}+1$, and evaluating a polynomial at -1,

$$\varphi(a) = a(-1) = \sum_i (-1)^i a_i \bmod q,$$

is a ring homomorphism from R_q to $\mathbb{Z}_q$. It turns each ring-LWE sample into a one-dimensional sample that is easy to solve. A server that receives one blinded TOPRF request $b = ar + a_x + 2e$ can test a password guess x' offline. It computes $c_j = \varphi(b_j) - \varphi(a_{x',j}) = \varphi(a_j)\varphi(r) + 2\varphi(e_j) + \varphi(a_{x,j} - a_{x',j})$ for each of the ℓ entries. For the right guess the last term vanishes, and $\varphi(r)$ and $\varphi(e_j)$ are sums of 595 small Gaussians, about $\sigma\sqrt{n} = 78.1$ in size. The server tries every ρ with $|\rho| \le 1500$ and keeps the guess if $c_j - \rho\,\varphi(a_j)$ is small in every entry. A wrong guess leaves the last term uniform in $\mathbb{Z}_q$, so no ρ fits.

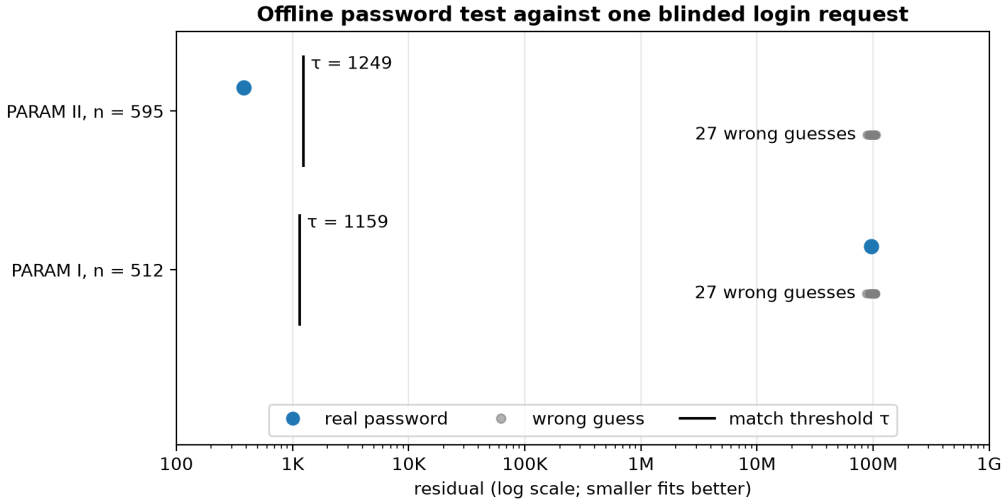


Figure 3: Residual of each of 28 guesses against one blinded login request. Below the threshold τ a guess matches. Drawn by `notebooks/06_param_ii_offline_guessing.ipynb`, cell 7.

Table 10: The offline test against one request. Source: `notebooks/06_param_ii_offline_guessing.ipynb`, cell 7.

parameter set	threshold tau	real password residual	smallest wrong-guess residual	largest wrong-guess residual	guesses matched
PARAM II, n = 595	1249	382	88243150	106898851	sunshine1
PARAM I, n = 512	1159	96673162	87519736	105468868	none

Notebook 06 runs the test over a dictionary of 28 common passwords that contains the real one (Figure 3 and Table 10). At PARAM II the real password leaves a residual of 382, below the threshold $\tau = 1249$, and every wrong guess leaves at least 88243150. The whole dictionary took 22.8 s. At PARAM I, $X^{512}+1$ has no root modulo q, and nothing matches: the real password's residual (96673162) is as large as the wrong guesses'. The attack breaks the resistance to offline guessing, the first attack in the paper's threat model, for PARAM II as we run it. The paper's security estimate for $n = 595$ probably assumes plain LWE, where this attack does not apply, so the flaw belongs to our reading of the parameters and is not proven of the paper. All other experiments therefore use PARAM I.

5.6.2 The key-update commitments hide nothing

Step 4 of the key update checks $H(F_{ji}) = \sum_k i^k H(\alpha_k)$, which requires H to be linear. We use $H(x) = A_H x \bmod q$ with a square matrix A_H, which is invertible modulo the prime q. The broadcast commitments

$h_k = H(\alpha_k)$ then reveal each α_k, hence every share F_{ji} and every key change $k'_i - k_i$, to anyone who sees them. The update no longer protects a key share that leaked in an earlier epoch. This holds for every linear H on uniform inputs. If H is injective it can be inverted. If it compresses, a dealer can add a vector from its kernel and change a share without changing H, so the commitment no longer binds. The shares still verify and K still stays the same, but a fix needs a hiding commitment with randomness, which the paper does not specify.

6 Proposed Scheme: A Counter-Based Search Index

To do (Person B). Write this section (handoff, Section 3.1): the two problems of QPASE's search, the design (keys, labels, index rows, counter table), and the changed phases (Login L4, Outsource, Retrieve) as `protocol` boxes like those in Section 3.

7 Analysis of the Proposed Scheme

To do (Person B). Write this section (handoff, Section 3.2): correctness, the leakage table (what a server learns), the forward-privacy argument, resilience against server attacks, and the costs we accept.

8 Evaluation of the Proposed Scheme

To do (Person B). Write this section (handoff, Section 3.3): the notebook 07 results, then search time against storage size with the generated figure and table from notebook 08.

9 Comparative Study

To do (Person B). Write this section (handoff, Section 3.4): the paper's scheme against the counter index, and QPASE against related password-based schemes.

10 Conclusion and Future Research Directions

To do (Person B). Write this section (handoff, Section 3.5): what the project showed, and what remains open.

References

[1] J. Jiang and D. Wang, "QPASE: Quantum-resistant password-authenticated searchable encryption for cloud storage," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 4231–4244, 2024.

[2] D. X. Song, D. Wagner, and A. Perrig, "Practical techniques for searches on encrypted data," in *Proc. IEEE Symposium on Security and Privacy*, pp. 44–55, 2000.

[3] R. Curtmola, J. Garay, S. Kamara, and R. Ostrovsky, "Searchable symmetric encryption: Improved definitions and efficient constructions," in *Proc. ACM Conference on Computer and Communications Security (CCS)*, pp. 79–88, 2006.

[4] A. Bagherzandi, S. Jarecki, N. Saxena, and Y. Lu, "Password-protected secret sharing," in *Proc. 18th ACM Conference on Computer and Communications Security (CCS)*, pp. 433–444, 2011.

[5] S. Jarecki, A. Kiayias, and H. Krawczyk, "Round-optimal password-protected secret sharing and T-PAKE in the password-only model," in *Proc. 20th International Conference on the Theory and Application of Cryptology and Information Security (ASIACRYPT)*, pp. 233–253, 2014.

[6] S. Jarecki, A. Kiayias, H. Krawczyk, and J. Xu, "Highly-efficient and composable password-protected secret sharing (or: How to protect your Bitcoin wallet online)," in *Proc. IEEE European Symposium on Security and Privacy*, pp. 276–291, 2016.

[7] S. Jarecki, A. Kiayias, H. Krawczyk, and J. Xu, "TOPPSS: Cost-minimal password-protected secret sharing based on threshold OPRF," in *Proc. Applied Cryptography and Network Security (ACNS)*, pp. 39–58, 2017.

[8] P. Das, J. Hesse, and A. Lehmann, "DPaSE: Distributed password-authenticated symmetric-key encryption, or how to get many keys from one password," in *Proc. ACM Asia Conference on Computer and Communications Security (AsiaCCS)*, pp. 682–696, 2022.

[9] L. Chen, K. Huang, M. Manulis, and V. Sekar, "Password-authenticated searchable encryption," *International Journal of Information Security*, vol. 20, no. 5, pp. 675–693, 2021.

[10] K. Huang, M. Manulis, and L. Chen, "Password authenticated keyword search," in *Proc. IEEE Symposium on Privacy-Aware Computing (PAC)*, pp. 129–140, 2017.

[11] P. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring," in *Proc. 35th Annual Symposium on Foundations of Computer Science (FOCS)*, pp. 124–134, 1994.

[12] O. Regev, "On lattices, learning with errors, random linear codes, and cryptography," *Journal of the ACM*, vol. 56, no. 6, pp. 1–40, 2009.

[13] P. Roy, S. Dutta, W. Susilo, and R. Safavi-Naini, "Password protected secret sharing from lattices," in *Proc. Applied Cryptography and Network Security (ACNS)*, pp. 442–459, 2021.

[14] J. Jiang, D. Wang, and G. Zhang, "QPause: Quantum-resistant password-protected data outsourcing for cloud storage," *IEEE Transactions on Services Computing*, 2024. early access.

[15] L. Ballard, S. Kamara, and F. Monrose, "Achieving efficient conjunctive keyword searches over encrypted data," in *Proc. International Conference on Information and Communications Security (ICICS)*, pp. 414–426, 2005.

[16] J. Jiang, D. Wang, G. Zhang, and Z. Chen, "Quantum-resistant password-based threshold single-sign-on authentication with updatable server private key," in *Computer Security, ESORICS*, pp. 295–316, 2022.

[17] V. Lyubashevsky, C. Peikert, and O. Regev, "On ideal lattices and learning with errors over rings," in *Advances in Cryptology, EUROCRYPT 2010*, vol. 6110 of *LNCS*, pp. 1–23, 2010.

[18] A. Banerjee and C. Peikert, "New and improved key-homomorphic pseudorandom functions," in *Advances in Cryptology, CRYPTO 2014*, pp. 353–370, 2014.

[19] J. Alwen, S. Krenn, K. Pietrzak, and D. Wichs, "Learning with rounding, revisited: New reduction, properties and applications," in *Advances in Cryptology, CRYPTO 2013*, pp. 57–74, 2013.

[20] M. Albrecht, A. Davidson, A. Deo, and N. P. Smart, "Round-optimal verifiable oblivious pseudorandom functions from ideal lattices," in *Proc. Public-Key Cryptography (PKC)*, pp. 261–289, 2021.

[21] J. Ding, X. Xie, and X. Lin, "A simple provably secure key exchange scheme based on the learning with errors problem." Cryptology ePrint Archive, Paper 2012/688, 2012.

[22] R. Bendlin, S. Krehbiel, and C. Peikert, "How to share a lattice trapdoor: Threshold protocols for signatures and (H)IBE," in *Proc. Applied Cryptography and Network Security (ACNS)*, pp. 218–236, 2013.

[23] H. Krawczyk, "Cryptographic extraction and key derivation: The HKDF scheme," in *Advances in Cryptology, CRYPTO 2010*, pp. 631–648, 2010.

[24] S. Bai, L. Ducas, and E. Kiltz, "CRYSTALS-Dilithium: Algorithm specifications and supporting documentation (version 3.1)," 2021.

[25] National Institute of Standards and Technology, "Module-lattice-based digital signature standard," Federal Information Processing Standards Publication FIPS 204, NIST, 2024.

[26] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proc. 28th Annual ACM Symposium on Theory of Computing (STOC)*, pp. 212–219, 1996.

[27] M. Dworkin, "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC," Special Publication 800-38D, NIST, 2007.

[28] M. Bellare, D. Pointcheval, and P. Rogaway, "Authenticated key exchange secure against dictionary attacks," in *Advances in Cryptology, EUROCRYPT 2000*, pp. 139–155, 2000.

[29] D. Wang, H. Cheng, P. Wang, X. Huang, and G. Jian, "Zipf's law in passwords," *IEEE Transactions on Information Forensics and Security*, vol. 12, no. 11, pp. 2776–2791, 2017.

A Reproducing the Numbers

Every measured value in this report is a LaTeX macro generated from a saved notebook output, the same cells that the repository's README cites. Nothing is typed in by hand. To reproduce the numbers from scratch:

1. Create the environment with `uv sync` in the repository root.
2. Run the unit tests with `.venv/bin/python -m unittest discover -s tests`.
3. Run all notebooks with `.venv/bin/python scripts/run_notebooks.py`, on an otherwise idle machine with at least three logical CPUs. On the test machine notebook 08 takes the longest by far (Table 5).
4. Regenerate the report's values with `.venv/bin/python report/tools/make_numbers.py`, then rebuild the report with `latexmk` in `report/`.

Timings will differ on another machine. Sizes, counts and pass or fail results will not.

Table 11 lists every generated value with the notebook cell that printed it. Cell *k* is the *k*-th code cell of the notebook.

Table 11: Every generated value and the notebook cell that printed it.

Macro and value	Source	Meaning
`\MachineCPU`: 13th Gen Intel(R) Core(TM) i5-13500, 20 logical CPUs	notebook 01, cell 13	processor and logical CPUs of the test machine
`\MachineMemory`: 15.3 GiB	notebook 01, cell 13	memory of the test machine
`\MachineOS`: Linux 7.2.11-arch1-1 (x86_64)	notebook 01, cell 13	operating system kernel of the test machine
`\MachinePython`: 3.14.9	notebook 01, cell 13	Python version

Macro and value	Source	Meaning
\MachinePackages: numpy 2.5.3, cryptography 50.0.2, dilithium-py 1.4.0, ipykernel 7.4.0, nbclient 0.11.0, matplotlib 3.11.2	notebook 01, cell 13	package versions
\ParamQ: 268435399	notebook 01, cell 2	modulus q of Table II
\ParamNOne: 512	notebook 01, cell 2	ring degree n of PARAM I
\ParamNTwo: 595	notebook 01, cell 2	ring degree n of PARAM II
\ParamEll: 28	notebook 01, cell 2	gadget width ell
\ParamP: 2	notebook 01, cell 2	rounding modulus p
\ParamSigma: 3.2	notebook 01, cell 2	standard deviation sigma of chi_sigma
\ParamL: 128	notebook 01, cell 2	input bit length L
\SizePublicVector: 57344 B	notebook 01, cell 2	size of each of a, a_0, a_1
\RingProductsExact: 3/3	notebook 01, cell 3	random ring products equal to the exact reference
\PrfOutputBits: 14336	notebook 01, cell 5	output bits of one BP PRF evaluation
\ToprfNoiseMax: 5904	notebook 01, cell 10	largest TOPRF noise coefficient in one run
\ToprfLoginNoiseMax: 9194	notebook 01, cell 10	largest difference between two TOPRF outputs
\QuarterQ: 67108849	notebook 01, cell 10	q/4, the extractor's noise bound
\KeyBytes: 1792 B	notebook 01, cell 11	size of K_u and of an HKDF output
\RegisterN: 3	notebook 02, cell 2	servers in notebook 02
\RegisterT: 2	notebook 02, cell 2	threshold in notebook 02
\MuLimit: 14	notebook 02, cell 2	attempt limit mu in notebook 02
\SizeToprfRequest: 57349 B	notebook 02, cell 4	TOPRF request to each server
\SizeToprfResponse: 114688 B	notebook 02, cell 4	Register TOPRF response from each server
\SizePkU: 1312 B	notebook 02, cell 5	size of pk_u (ML-DSA-44 public key)
\SizeRegisterKeyMessage: 3109 B	notebook 02, cell 5	Register step 2 message
\SizeLoginResponse: 117792 B	notebook 02, cell 6	Login L1 response from each server
\SizeLoginProof: 2457 B	notebook 02, cell 6	Login L2 proof to each server
\LoginsIdentical: 10/10	notebook 02, cell 10	repeated logins with byte-identical K_u
\LoginMedian: 2.19 s	notebook 02, cell 10	median time of one Login with t servers
\SizeSigCt: 2420 B	notebook 03, cell 4	size of sig_Ct (ML-DSA-44 signature)
\SizeCt: 4256 B	notebook 03, cell 4	size of the index ciphertext Ct
\SizeOutsourceRequest: 6858 B	notebook 03, cell 4	Outsource request for a 149-byte document
\SizeRetrieveRequest: 4217 B	notebook 03, cell 9	Retrieve request to each server
\RTwoAllServers: 1.42 s	notebook 03, cell 10	time of R2 on all 3 servers, 7 items
\RecoveredItems: [1, 3, 6]	notebook 03, cell 12	items recovered for the keyword medical
\SizeKeyShare: 2048 B	notebook 04, cell 4	size of a key share k_i
\SizeUpdateShare: 6544 B	notebook 04, cell 5	payload of one key update share
\SizeUpdateAll: 58896 B	notebook 04, cell 5	all shares of one key update, N = 3
\UpdateSharesAccepted: 9 of 9	notebook 04, cell 6	key update shares that pass step 4
\UpdateEpochsAfterTwo: [2, 2, 2]	notebook 04, cell 11	epochs after two key updates
\AxMedian: 492 ms	notebook 05, cell 3	median time of one PRF chain a_x

Macro and value	Source	Meaning
\CostChecks: 270	notebook 05, cell 12	timed runs whose result was checked
\CostFailures: 0	notebook 05, cell 12	failed checks over the timed runs
\ParamIISigmaSqrtN: 78.1	notebook 06, cell 2	spread of phi(r) at PARAM II
\ParamIIRadius: 1500	notebook 06, cell 2	search radius of the offline test
\ParamIITau: 1249	notebook 06, cell 2	match threshold at PARAM II
\ParamIIGuesses: 28	notebook 06, cell 3	dictionary size
\ParamIIAttackTime: 22.8 s	notebook 06, cell 3	time to test the dictionary
\ParamIITrueResidual: 382	notebook 06, cell 4	residual of the real password at PARAM II
\ParamIIMinWrong: 88243150	notebook 06, cell 4	smallest wrong-guess residual at PARAM II
\ParamIIMaxWrong: 106898851	notebook 06, cell 4	largest wrong-guess residual at PARAM II
\ParamITau: 1159	notebook 06, cell 5	match threshold at PARAM I
\ParamIReal: 96673162	notebook 06, cell 5	residual of the real password at PARAM I
\ParamISmallest: 87519736	notebook 06, cell 5	smallest residual of any guess at PARAM I
\CiLabelBytes: 32 B	notebook 07, cell 4	size of a counter-index label
\CiEntryBytes: 108 B	notebook 07, cell 4	size of a counter-index entry
\CiOutsourceRequestBytes: 2742 B	notebook 07, cell 4	counter-index Outsource request for item 1
\CiSealedBytes: 1072 B	notebook 07, cell 5	sealed counter table
\CiUploadBytes: 3497 B	notebook 07, cell 5	counter table upload to each server
\CiSearchRequestBytes: 2521 B	notebook 07, cell 9	label search request for c_w = 3
\CiFetchRequestBytes: 2425 B	notebook 07, cell 12	file fetch request for one item
\CiLookups: [3, 3, 3]	notebook 07, cell 13	lookups per server in one search
\CiReplayItems: [1, 3, 6]	notebook 07, cell 14	items a replayed old request finds
\CiNewSearchItems: [1, 3, 6, 8]	notebook 07, cell 14	items a new search finds
\CiNewDeviceVersions: [8, 8]	notebook 07, cell 16	counter table versions a new device gets
\ScalingChecks: 144	notebook 08, cell 6	timed scaling runs whose result was checked
\ScalingFailures: 0	notebook 08, cell 6	failed checks over the scaling runs
\ScalingLookups: [3]	notebook 08, cell 6	lookups per server in every run
\CostRegisterUser: 1.39 s	notebook 05, cell 13	median of RegisterUser in notebook 05
\CostRegisterServer: 1.19 ms	notebook 05, cell 13	median of RegisterServer in notebook 05
\CostLoginUser: 1.90 s	notebook 05, cell 13	median of LoginUser in notebook 05
\CostLoginServer: 5.94 ms	notebook 05, cell 13	median of LoginServer in notebook 05
\CostOutsourceUser: 994 ms	notebook 05, cell 13	median of OutsourceUser in notebook 05
\CostOutsourceServer: 4.95 ms	notebook 05, cell 13	median of OutsourceServer in notebook 05
\CostRetrieveUser: 1.50 s	notebook 05, cell 13	median of RetrieveUser in notebook 05
\CostRetrieveServer: 1000 ms	notebook 05, cell 13	median of RetrieveServer in notebook 05
\CostKeyUpdateServer: 108 ms	notebook 05, cell 13	median of KeyUpdateServer in notebook 05
\CostKeyUpdateRound: 326 ms	notebook 05, cell 13	median of KeyUpdateRound in notebook 05

Macro and value	Source	Meaning
\CostSignMedian: 19.8 ms	notebook 05, cell 13	median ML-DSA-44 signature
\CostVerifyMedian: 4.79 ms	notebook 05, cell 13	median ML-DSA-44 verification
\CostAesMedian: 0.021 ms	notebook 05, cell 13	median AES-256-GCM encryption of 100 KB
\TrafficRegister: 525462 B	notebook 05, cell 13	total bytes of Register, N = 3
\TrafficLogin: 355260 B	notebook 05, cell 13	total bytes of Login, N = 3
\TrafficOutsource: 106709 B	notebook 05, cell 13	total bytes of Outsource, N = 3
\TrafficRetrieve: 221213 B	notebook 05, cell 13	total bytes of Retrieve, N = 3
\TrafficKeyUpdate: 39264 B	notebook 05, cell 13	total bytes of KeyUpdate, N = 3
\WallOne: 9 s	notebook 01, cell 13	wall time of notebook 01
\WallTwo: 35 s	notebook 02, cell 13	wall time of notebook 02
\WallThree: 18 s	notebook 03, cell 17	wall time of notebook 03
\WallFour: 21 s	notebook 04, cell 13	wall time of notebook 04
\WallFive: 141 s	notebook 05, cell 14	wall time of notebook 05
\WallSix: 43 s	notebook 06, cell 8	wall time of notebook 06
\WallSeven: 22 s	notebook 07, cell 18	wall time of notebook 07
\WallEight: 828 s	notebook 08, cell 8	wall time of notebook 08
\PeakOne: 117 MiB	notebook 01, cell 13	peak memory of notebook 01
\PeakTwo: 118 MiB	notebook 02, cell 13	peak memory of notebook 02
\PeakThree: 114 MiB	notebook 03, cell 17	peak memory of notebook 03
\PeakFour: 121 MiB	notebook 04, cell 13	peak memory of notebook 04
\PeakFive: 148 MiB	notebook 05, cell 14	peak memory of notebook 05
\PeakSix: 135 MiB	notebook 06, cell 8	peak memory of notebook 06
\PeakSeven: 117 MiB	notebook 07, cell 18	peak memory of notebook 07
\PeakEight: 574 MiB	notebook 08, cell 8	peak memory of notebook 08
\ScalingPaperMin: 510 ms	notebook 08, cell 7 (derived from the table)	fastest paper slowest-server median over the grid
\ScalingPaperMax: 721 ms	notebook 08, cell 7 (derived from the table)	slowest paper slowest-server median over the grid
\ScalingIndexMin: 4.61 ms	notebook 08, cell 7 (derived from the table)	fastest counter-index slowest-server median over the grid
\ScalingIndexMax: 4.85 ms	notebook 08, cell 7 (derived from the table)	slowest counter-index slowest-server median over the grid
\ScalingSpeedupMin: 109	notebook 08, cell 7 (derived from the table)	smallest speed-up at the slowest server
\ScalingSpeedupMax: 152	notebook 08, cell 7 (derived from the table)	largest speed-up at the slowest server
\ScalingSpeedupMedian: 114	notebook 08, cell 7 (derived from the table)	median speed-up at the slowest server
\UnitTests: 184	README.md, Results (the unittest run)	unit tests that passed on the test machine